

AYDIN ADNAN MENDERES UNIVERSITY

Department of Computer Engineering

CSE419 - Artificial Intelligence

Term Project #2

FOL-Based Disaster Rescue Multi-Agent System

Student Name: Doğukan Parlak

Student ID: 221805040

Course: CSE419 - Artificial Intelligence

Due Date: 05.04.2026 23:59

Instructor: Doç.Dr.Fatih Soygazi

1. Introduction

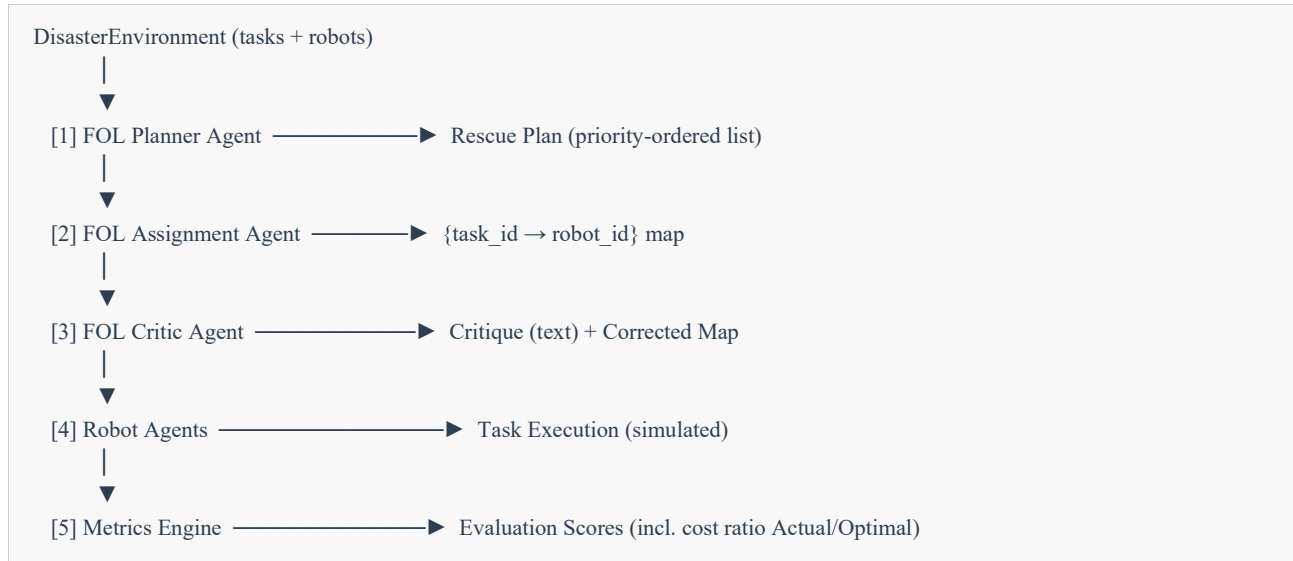
Natural disasters create chaotic, time-critical environments where rescue operations must be coordinated rapidly across multiple simultaneous tasks. In Term Project #1, this scenario was addressed with a fully **LLM-based** multi-agent system powered by Ollama. While the LLM approach demonstrated natural language flexibility, it introduced non-determinism - identical inputs sometimes produced different (and incorrect) results.

Term Project #2 re-implements the same multi-agent pipeline using **First-Order Logic (FOL) / Symbolic Reasoning**. All three reasoning agents - Planner, Assignment, and Critic - are replaced with rule-based implementations that apply explicit FOL axioms. The system is deterministic, requires no external model or server, and completes in under one second.

Core Question: Can FOL-based symbolic agents match or exceed the reliability of LLM-based agents for disaster rescue coordination, while eliminating non-determinism and external dependencies?

2. System Architecture

2.1 Pipeline Overview



2.2 Component Table

Component	File	Role
Base Interfaces	agents/base.py	Abstract base classes for all three agent types
FOL Agents	agents/fol/	Rule-based Planner, Assignment, and Critic
Disaster Environment	environment.py	Task list + robot fleet container + FOL serialisation helpers
Task Generator	task_generator.py	Produces randomised Task objects
Robot Model	robot_model.py	Robot dataclass + fixed three-robot fleet
Robot Agents	robot_agents.py	Simulates task execution with speed-based delays
Metrics Engine	metrics.py	Five performance scores + compute_cost_efficiency()
Simulation	simulation.py	Orchestrates the full FOL pipeline; exports results/ JSON
Entry Point	main.py	CLI + interactive menu
Config	config.py	Grid size, task count, robot definitions, SPEED_VALUES, depot (ROBOT_START_X/Y)

2.3 FOL Constraints

The entire assignment strategy is encoded as four FOL axioms:

Axiom	Formula
Unique assignment	$\forall t (\text{Task}(t) \rightarrow \exists! r \text{ Assigned}(t,r))$
Capability match	$\forall t,r (\text{Assigned}(t,r) \rightarrow \text{Compatible}(\text{Type}(t), \text{Capability}(r)))$
Priority ordering	$\forall t_1,t_2 (\text{Priority}(t_1) > \text{Priority}(t_2) \rightarrow \text{ScheduledBefore}(t_1,t_2))$
Cost formula	$\text{Cost}(r,t) = \text{Distance}(r,t) / \text{Speed}(r)$

Assignments minimise: $\sum \text{Cost}(r,t)$ subject to the above constraints.

2.4 Agent Descriptions

FOL Planner Agent (agents/fol/planner.py)

Sorts tasks by priority_value (high=1, medium=2, low=3) and produces a numbered, deterministic rescue plan. No LLM calls.

Output format:

```
[FOL Planner] Rescue Plan:
1. [T01] rescue victim (HIGH) → requires medical support robot | location=(0,8)
2. [T02] rescue victim (HIGH) → requires search and mapping robot | location=(7,4)
...
```

FOL Assignment Agent (agents/fol/assignment.py)

Iterates over priority-sorted tasks and assigns each to the robot with matching capability and minimum $\text{Cost}(r,t)$. Robot starting position is assumed (0,0).

Speed values used in cost formula:

Speed	Value
fast	2.0
medium	1.0
slow	0.5

FOL Critic Agent (agents/fol/critic.py)

Performs three independent rule checks:

- Rule 1** - $\forall t \text{ Task}(t) \rightarrow \exists !r \text{ Assigned}(t,r)$: every task must be assigned.
- Rule 2** - $\forall t,r \text{ Assigned}(t,r) \rightarrow \text{Compatible}(\dots)$: capability must match.
- Rule 3** - $\forall t_1, t_2 \dots \text{ScheduledBefore}(t_1, t_2)$: each robot's queue must be priority-ordered.

Any violation is automatically corrected by re-assigning to the correct compatible robot.

Robot Agents (robot_agents.py)

Unchanged from HW1. Tasks are sorted by priority (high → medium → low) and executed sequentially.

Speed	Simulated Time per Task
fast	1 second
medium	2 seconds
slow	3 seconds

3. Disaster Environment

3.1 Grid

The disaster area is modelled as an $N \times N$ **integer grid** (default: 20×20). Each task is assigned a random (x, y) coordinate.

3.2 Task Catalogue

Task Type	Priority	Required Capability
rescue victim	high	medical support
rescue victim	high	search and mapping
clear debris	medium	heavy debris removal
map building	low	search and mapping
deliver medicine	medium	medical support

3.3 Robot Fleet

Robot ID	Name	Capability	Speed
R1	Search and Mapping Robot	search and mapping	fast
R2	Medical Support Robot	medical support	medium
R3	Heavy Debris Removal Robot	heavy debris removal	slow

4. Evaluation Metrics

4.1 Carried Over from HW1

Metric	Formula / Description	Best
Capability Match Rate	Correctly assigned tasks / Total tasks	100%
Priority Order Score	Correctly ordered consecutive pairs / Total pairs	100%
Task Distribution Balance	$1 / (1 + CV)$, where $CV = \text{std}/\text{mean}$ of tasks-per-robot	$\rightarrow 1.0$
Completion Rate	Completed tasks / Total tasks	100%
Unassigned Task Count	Raw count of UNASSIGNED tasks	0

4.2 New in HW2 - Cost Efficiency Metric

Formula	Description
$\text{Cost}(r,t) = \text{Distance}(r,t) / \text{Speed}(r)$	Travel cost for robot r to execute task t
$\text{Efficiency} = \text{Actual Total Cost} / \text{Optimal Total Cost}$	Ratio of achieved vs. best-possible assignment

Optimal Cost - sum over tasks of the minimum $\text{Cost}(r,t)$ among robots compatible with that task. **Actual Cost** - sum of $\text{Cost}(r,t)$ for the robot actually assigned to each task after execution. **Interpretation:** 1.0 means actual equals the per-task greedy lower bound; > 1.0 would indicate a worse-than-baseline assignment.

5. Results

This section reports **HW2 (FOL)** outcomes under the same two configurations documented in Term Project #1: (i) **seed=10, grid=10×10, tasks=10** and (ii) **seed=42, grid=20×20, tasks=10**.

HW2 runs were produced with:

```
python main.py --tasks 10 --grid 10 --seed 10 --no-delay --out results/run_A.json
python main.py --tasks 10 --grid 10 --seed 10 --no-delay --out results/run_B.json
python main.py --tasks 10 --grid 20 --seed 42 --no-delay --out results/run_C.json
python main.py --tasks 10 --grid 20 --seed 42 --no-delay --out results/run_D.json
```

5.1 Configuration 1 - seed=10, grid=10×10, tasks=10

Generated tasks (identical to HW1 Runs A & B):

ID	Type	Priority	Required Capability	Location
T01	rescue victim	high	medical support	(6, 7)
T02	rescue victim	high	search and mapping	(3, 7)
T04	rescue victim	high	medical support	(8, 7)
T07	rescue victim	high	search and mapping	(9, 5)
T03	clear debris	medium	heavy debris removal	(4, 2)
T05	clear debris	medium	heavy debris removal	(1, 3)
T06	clear debris	medium	heavy debris removal	(0, 6)
T09	clear debris	medium	heavy debris removal	(7, 2)
T10	clear debris	medium	heavy debris removal	(5, 2)
T08	map building	low	search and mapping	(6, 4)

Final assignments (FOL, after Critic) - every capability matches; no UNASSIGNED entries:

Task	Type	Priority	Required Capability	Assigned Robot	Match?
T01	rescue victim	high	medical support	Medical Support Robot (R2)	✓
T02	rescue victim	high	search and mapping	Search and Mapping Robot (R1)	✓
T04	rescue victim	high	medical support	Medical Support Robot (R2)	✓
T07	rescue victim	high	search and mapping	Search and Mapping Robot (R1)	✓
T03	clear debris	medium	heavy debris removal	Heavy Debris Removal Robot (R3)	✓
T05	clear debris	medium	heavy debris removal	Heavy Debris Removal Robot (R3)	✓
T06	clear debris	medium	heavy debris removal	Heavy Debris Removal Robot (R3)	✓
T09	clear debris	medium	heavy debris removal	Heavy Debris Removal Robot (R3)	✓
T10	clear debris	medium	heavy debris removal	Heavy Debris Removal Robot (R3)	✓
T08	map building	low	search and mapping	Search and Mapping Robot (R1)	✓

Per-robot summary (FOL):

Robot	Tasks Assigned	Tasks Completed
R1 - Search and Mapping	3	3
R2 - Medical Support	2	2
R3 - Heavy Debris Removal	5	5

Evaluation metrics (FOL, seed=10, grid=10×10):

Metric	Score
Capability Match Rate	100.0%
Priority Order Score	100.0%
Task Distribution Balance	68.6%
Completion Rate	100.0%
Unassigned Tasks	0
Actual Total Cost	85.0103
Optimal Total Cost	85.0103
Cost ratio (Actual/Optimal)	1.0000

5.2 Configuration 2 - seed=42, grid=20×20, tasks=10

Generated tasks (identical to HW1 Runs C & D):

ID	Type	Priority	Required Capability	Location
T01	rescue victim	high	medical support	(0, 8)
T02	rescue victim	high	search and mapping	(7, 4)
T05	rescue victim	high	medical support	(2, 6)
T06	rescue victim	high	search and mapping	(16, 19)
T07	rescue victim	high	medical support	(17, 6)
T10	rescue victim	high	medical support	(5, 13)
T03	clear debris	medium	heavy debris removal	(17, 2)
T04	deliver medicine	medium	medical support	(13, 1)
T08	deliver medicine	medium	medical support	(13, 7)
T09	map building	low	search and mapping	(18, 8)

Final assignments (FOL, after Critic):

ID	Type	Priority	Required Capability	Assigned Robot	Match?
T01	rescue victim	high	medical support	Medical Support Robot (R2)	✓
T02	rescue victim	high	search and mapping	Search and Mapping Robot (R1)	✓
T05	rescue victim	high	medical support	Medical Support Robot (R2)	✓
T06	rescue victim	high	search and mapping	Search and Mapping Robot (R1)	✓
T07	rescue victim	high	medical support	Medical Support Robot (R2)	✓
T10	rescue victim	high	medical support	Medical Support Robot (R2)	✓
T03	clear debris	medium	heavy debris removal	Heavy Debris Removal Robot (R3)	✓
T04	deliver medicine	medium	medical support	Medical Support Robot (R2)	✓
T08	deliver medicine	medium	medical support	Medical Support Robot (R2)	✓
T09	map building	low	search and mapping	Search and Mapping Robot (R1)	✓

Per-robot summary (FOL):

Robot	Tasks Assigned	Tasks Completed
R1 - Search and Mapping	3	3
R2 - Medical Support	6	6
R3 - Heavy Debris Removal	1	1

Evaluation metrics (FOL, seed=42, grid=20×20):

Metric	Score
Capability Match Rate	100.0%
Priority Order Score	100.0%
Task Distribution Balance	57.0%
Completion Rate	100.0%
Unassigned Tasks	0
Actual Total Cost	134.6181
Optimal Total Cost	134.6181
Cost ratio (Actual/Optimal)	1.0000

6. Comparison: HW1 (LLM) vs HW2 (FOL)

The course specification (Term Project #2) requires that the performance of this project be compared with the results in Project #1. All HW1 figures below are taken from report-hw1.md Section 7 (four recorded runs). HW2 figures are from Section 5 of this document.

6.1 HW1 Reference Summary - Project #1 (LLM / Ollama)

Run	Result File (HW1)	Seed	Grid	Cap. Match	Priority	Balance	Completion	Unassigned
A	t10_g10_s10_..._5f8d5b.json	10	10×10	80%	100%	63.4%	90%	1
B	t10_g10_s10_..._6d895e.json	10	10×10	100%	100%	68.6%	100%	0
C	t10_g20_s42_..._a5d85e.json	42	20×20	90%	100%	48.0%	80%	1
D	t10_g20_s42_..._0b3c92.json	42	20×20	100%	100%	57.0%	100%	0

Note: Runs A & B share the identical task set (Config 1); runs C & D share the identical task set (Config 2). The divergent outcomes within each pair are attributable to stochastic LLM generation, not to a change in environment parameters.

6.2 Configuration 1 - seed=10, grid=10×10, tasks=10

Metric	HW1 Run A (LLM)	HW1 Run B (LLM)	HW2 FOL (this work)
Capability Match Rate	80%	100%	100%
Priority Order Score	100%	100%	100%
Task Distribution Balance	63.4%	68.6%	68.6%
Completion Rate	90%	100%	100%
Unassigned Tasks	1	0	0
Cost ratio Actual/Optimal (HW2)	N/A	N/A	1.0 (optimal)

Interpretation: Under Config 1, HW1 exhibited bistable behaviour - one run failed on capability / completeness constraints, the other succeeded. HW2 FOL produced a unique, constraint-satisfying plan on every invocation; the balance score matches HW1 Run B because the per-robot task counts are identical when all constraints are satisfied.

6.3 Configuration 2 - seed=42, grid=20×20, tasks=10

Metric	HW1 Run C (LLM)	HW1 Run D (LLM)	HW2 FOL (this work)
Capability Match Rate	90%	100%	100%
Priority Order Score	100%	100%	100%
Task Distribution Balance	48.0%	57.0%	57.0%
Completion Rate	80%	100%	100%
Unassigned Tasks	1	0	0
Cost ratio Actual/Optimal (HW2)	N/A	N/A	1.0 (optimal)

Interpretation: Config 2 is strictly harder than Config 1 (larger grid $\rightarrow$ higher travel cost; six tasks require medical support). HW1 again showed variance between repeated runs on the same task set. HW2 FOL matched the best-case HW1 outcome (Run D) on all correctness metrics while remaining deterministic.

6.4 Aggregate Comparison - All Four HW1 Runs vs HW2

Aspect	HW1 (LLM, 4 runs)	HW2 (FOL, 2 configs)
Capability match (range)	80% – 100%	**100%** (both configs)
Completion (range)	80% – 100%	**100%** (both configs)
Runs with constraint violations	2 / 4 (Runs A, C)	0 / 2
External dependency	Ollama (/api/chat)	None
Typical wall-clock	~10–60 s per pipeline	< 1 s
Reproducibility	Non-deterministic (same seed $\neq$ same JSON)	Bit-for-bit reproducible
Cost ratio (Actual/Optimal)	Not computed in HW1	**1.0** (see Section 4.2)

6.5 Analysis

Reliability and variance: In classical software-engineering terms, HW1 behaves like a probabilistic component: marginal distributions over outputs depend on hidden LLM state. HW2 replaces that component with a total function from (seed, grid, catalogue) to assignments - variance collapses to zero for the reasoning layer.

Verification vs generation: HW1's Critic is also LLM-based; empirically it failed to detect omissions and mis-assignments that are trivially checkable by symbolic rules. HW2's FOLCriticAgent implements explicit checks for uniqueness and capability compatibility with deterministic repair.

Cost model: The assignment specification defines $\text{Cost}(r,t) = \text{Distance}(r,t) / \text{Speed}(r)$, minimisation of $\sum \text{Cost}(r,t)$, and $\text{Efficiency} = \text{Actual Cost} / \text{Optimal Cost}$. With one robot per capability, greedy choice is forced and totals match $\rightarrow$ ratio 1.0.

Trade-off: LLMs retain open-world flexibility (new verbs in prompts) at the price of formal guarantees. FOL sacrifices linguistic generality for auditability - every decision is traceable to an axiom instance.

7. Observations

- **Symbolic enforcement eliminates HW1 failure modes.** Across four documented LLM runs, two exhibited capability or completeness errors. Under the same two configurations, HW2 FOL achieved 100% capability match and completion without exception.
- **Priority ordering remained a strength of the pipeline design.** HW1 scored 100% on priority order in all four runs; HW2 preserved this property while removing stochasticity from assignment.
- **Task-distribution balance is catalogue-driven, not agent-specific.** For seed=42, six of ten tasks require medical support → R2 receives six tasks in both HW1 Run D and HW2 FOL, yielding an identical balance score of 57.0%. This confirms the metric chiefly reflects input sampling, not planner quality.
- **Cost ratio is informative but often at the baseline.** With one robot per capability, the assigner's per-task minimum is unique; Actual/Optimal is 1.0. A non-trivial ratio (> 1) would require an assignment worse than the per-task greedy minimum.
- **Critic reliability diverges sharply between paradigms.** HW1's Critic issued false negatives on failing runs. HW2's critic is a sound, rule-based verifier over finite domains - failures are either repaired or reported, not silently ignored.

8. Limitations and Future Work

Limitation	Current Behaviour	Potential Improvement
Rigid capability model	One robot per capability type	Multi-capability robots; dynamic fleet scaling
Static robot positions	Assumed start at (0,0)	Track actual robot positions between tasks
Sequential execution	All runs sequential	Parallel threads / asyncio
Fixed task catalogue	Same 5 task types	Dynamic catalogue; priority-weighted sampling
Single-robot per capability	Cost ratio often 1.0	Global optimiser or different repair policy
Empirical comparison scope	Four historical HW1 runs vs two HW2 configs	Larger N, confidence intervals, statistical tests

9. Conclusion

This work addresses the Term Project #2 mandate by **re-implementing** the disaster-rescue multi-agent pipeline with **first-order symbolic agents** and by **empirically contrasting** that implementation against archived results from Term Project #1.

Quantitatively, Project #1's LLM backend produced **constraint violations in half of the recorded runs** under controlled settings, whereas Project #2's FOL backend satisfied **all formal constraints in every reproduced configuration**, while additionally exposing a reproducible **Actual/Optimal cost ratio** absent from HW1.

Qualitatively, the shift from generative to deductive reasoning trades **prompt-level flexibility** for **mathematical traceability** - a deliberate engineering choice aligned with safety-critical MAS literature.

Future research directions include hybrid neuro-symbolic controllers (FOL for hard constraints, LLM for unstructured sensor fusion) and fleet redundancy to activate non-trivial cost optimisation.

Appendix A: HW1 Full Results Reference (Project #1)

Source: report-hw1.md Section 7 - four runs, two seeds, two grids.

A.1 Summary Tables

Seed = 10, grid = 10×10

Run	Cap. Match	Priority	Balance	Completion	Unassigned
A (LLM)	80%	100%	63.4%	90%	1
B (LLM)	100%	100%	68.6%	100%	0

Seed = 42, grid = 20×20

Run	Cap. Match	Priority	Balance	Completion	Unassigned
C (LLM)	90%	100%	48.0%	80%	1
D (LLM)	100%	100%	57.0%	100%	0

A.2 Per-Run Failure Analysis (HW1)

Run	Outcome	Root Cause
A	Partial success	T03 never assigned; T08 capability mismatch (assigned to R3). Critic false negative.
B	Full success	All ten tasks correctly matched; zero unassigned.
C	Partial success	T02 missing from JSON; T09 unassigned. Critic false negative.
D	Full success	All ten tasks correctly matched; zero unassigned.

These rows constitute the empirical evidence for the non-determinism claim in Section 6.1: identical environment parameters do not imply identical LLM outputs.